



正点原子公司名称 : 广州市星翼电子科技有限公司

原子哥在线教学平台 : www.yuanzige.com

开源电子网 / 论坛 : <http://www.openedv.com/forum.php>

正点原子淘宝店铺 : <https://openedv.taobao.com>

正点原子官方网站 : www.alientek.com

正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。

请下载原子哥 APP, 数千讲视频免费学习, 更快更流畅。



扫码关注正点原子公众号



扫码下载“原子哥”APP

目录

SPL 启动流程分析	3
1.1 概述	3
1.2 整体启动流程概述	4
1.3 board_init_f_boot_flags 函数	7
1.4 board_init_f 函数	7
1.4.1 spl_early_init 函数	9
1.5 board_init_r 函数	11
1.5.1 board_boot_order 函数	13
1.5.2 boot_from_devices 函数	14
1.5.3 spl_invoke_atf 函数	15
1.6 spl_mmc_load_image 函数	17
1.6.1 mmc_load_image_raw_partition 函数	20
1.6.2 A/B 分区的处理	20
1.6.3 mmc_load_image_raw_sector 函数	24
1.6.4 spl_load_simple_fit 函数	25
1.6.5 spl_internal_load_simple_fit 函数	26
1.6.6 spl_load_fit_image 函数	31

SPL 启动流程分析

1.1 概述

首先了解下 RK 平台启动它的一个流程:

启动流程:

BOOTROM => TPL(DDR bin) => SPL(miniload) => TRUST => U-BOOT => KERNEL

BOOTROM 就是 RK 芯片内存固化的程序, 由该程序加载 TPL 程序, TPL 程序的作用就是去初始化外挂的 DDR, 所以 TPL 程序肯定是在 SRAM (片内) 中运行的; TPL 运行完成之后, DDR 就已经初始化成功了, 此时它会去启动 SPL, SPL 其实是一个在 U-Boot 之前运行的一段程序, 当然 U-Boot 并不是由 SPL 启动的, 在我们的 RK 平台, U-Boot 通常是由 TRUST 去启动的; 而 TRUST 则由 SPL 负责启动。

对于我们的 RK3568 平台, 会将 TPL 和 SPL 做成一个 MiniLoaderAll.bin 的镜像文件, 所以这个 MiniLoaderAll.bin 文件就是集成了 TPL 和 SPL; 而 TRUST 会和 u-boot 镜像集成成一个 uboot.img 镜像, 所以对于 RK3568, 它并没有单独提供 trust.img 镜像, 而是集成到了 uboot.img 镜像。

RK 平台提供了两种 TPL 和 SPL: 闭源和开源。

首先闭源的 TPL 和 SPL, 只是提供了相应的二进制镜像文件, 并没有提供其源码, 这些二进制镜像文件都在 RK SDK 包目录下的 rkbin 文件夹中可找到, 对于 RK3568 平台来说, 在 rkbin/bin/rk35 目录下, 如下所示:

```
lientek@android:~/hdd/rk3568-a/rkbin/bin/rk35$ ls
FlashHead.bin          rk3568_ddr_630MHz_v1.09.bin  rk3568_bl31_ultra_v2.06.elf  rk3568_ddr_528MHz_v1.09.bin  rk356x_spl_nand_v1.07.bin
rk3566_ddr_1056MHz_ultra_print_off_v1.07.bin  rk3566_ddr_780MHz_ultra_print_off_v1.07.bin  rk3568_bl31_v1.27.elf       rk3568_ddr_630MHz_v1.09.bin  rk356x_spl_v1.11.bin
rk3566_ddr_1056MHz_ultra_v1.07.bin            rk3566_ddr_780MHz_ultra_v1.07.bin            rk3568_bl32_v1.05.bin       rk3568_ddr_780MHz_v1.09.bin  rk356x_usbplug_nand_v1.04.bin
rk3566_ddr_1056MHz_v1.09.bin                 rk3566_ddr_780MHz_v1.09.bin                 rk3568_ddr_1056MHz_v1.09.bin rk3568_ddr_920MHz_v1.09.bin  rk356x_usbplug_v1.10.bin
rk3566_ddr_528MHz_ultra_print_off_v1.07.bin   rk3566_ddr_920MHz_ultra_print_off_v1.07.bin   rk3568_ddr_1184MHz_v1.09.bin rk3568_ramboot_null0.bin    UsbHead.bin
rk3566_ddr_528MHz_ultra_v1.07.bin            rk3566_ddr_920MHz_ultra_v1.07.bin            rk3568_ddr_1332MHz_v1.09.bin rk3568_ramboot_null1.bin
rk3566_ddr_528MHz_v1.09.bin                 rk3566_ddr_920MHz_v1.09.bin                 rk3568_ddr_1560MHz_v1.09.bin rk3568_ramboot_v1.08.bin
```

可以看到这个目录下有很多的 rk3568_ddr_xxx 的镜像文件, 这些其实就是 TPL 镜像; 这些 rk356x_spl_x 的文件便是 SPL 镜像文件; 这就是 RK 平台所提供的闭源的镜像, 只有二进制镜像文件、没有其源码。

除了使用闭源的 TPL+SPL 外, 我们还可以使用开源的 TPL+SPL, 其实这个通过 u-boot 代码即可编译出来, 在 RK 提供的 u-boot 源码中, 直接使用已经配置好的脚本编译 (譬如: ./make.sh rk3568) 成功之后便会生成 TPL 镜像和 SPL 镜像, TPL 镜像是 tpl/u-boot-tpl.bin, SPL 镜像是 spl/u-boot-spl.bin。

所以根据 TPL、SPL 是否开源, 可以有两套启动方式:

RK平台根据前级Loader代码是否开源, 目前有两套启动方式:

```
// 前级loader闭源
BOOTROM => ddr bin => Miniload => TRUST => U-BOOT => KERNEL

// 前级loader开源
BOOTROM => TPL => SPL => TRUST => U-BOOT => KERNEL
```

TPL 相当于 ddr bin, SPL 相当于 miniload。TPL+SPL 的组合实现了跟 RK 闭源 ddr.bin+miniload 一致的功能, 可相互替换。

上图中的 ddr bin 指的就是 RK 提供的闭源 TPL, 上图中的 Miniload 指的就是 RK 提供的闭源 SPL。

当然也可以混着使用,譬如笔者喜欢使用闭源 TPL + 开源 SPL 来实现启动,需要注意的是,对于 RK3568 平台,开源 TPL 不可用,这个笔者测试过,使用开源的 TPL 无法启动,后分析其代码得知,并未实现 DDR 初始化相关代码,估计是 RK 还没有针对 3568 做支持。

上面还涉及到一个 TRUST 镜像,这个镜像涉及到 ARM 安全相关 (ARM TrustZone), RK 提供的文档也有相关介绍:

目前用于 ARM TrustZone 技术的开源项目中,使用比较广泛的有 ARM Trusted Firmware 和 OP-TEE OS[2],它们都是针对 ARM 芯片给出的底层固件开源项目,二者之间可以配合使用或单独使用。

所以从 ARM TrustZone 角度上分析 RK 平台的启动流程:

ARM Trusted Firmware 的体系架构里将整个系统分成四种安全等级,分别为: EL0、EL1、EL2、EL3。

将整个安全启动的流程阶段定义为: BL1、BL2、BL31、BL32、BL33,其中 ARM Trusted Firmware 自身的源代码里提供了 BL1、BL2、BL31 的功能。Rockchip 平台仅使用了其中的 BL31 的功能, BL1 和 BL2 我们有自己的一套实现方案。所以在 Rockchip 平台上我们一般也可以“默认” ARM Trusted Firmware 指的就是 BL31,而 BL32 使用的则是 OP-TEE OS。

如果把上述这种阶段定义映射到 Rockchip 的平台各级固件上,对应关系为: Maskrom (BL1)、Loader (BL2)、Trust (BL31: ARM Trusted Firmware + BL32: OP-TEE OS)、U-Boot (BL33)。

Android 系统的固件启动顺序:

```
Maskrom -> Loader -> Trust -> U-Boot -> kernel -> Android
```

从上面我们需要知道一些信息, BL31 指的是 ARM Trusted Firmware 固件代码, BL32 指的是 OPTEE, BL33 指的就是 u-boot,后面分析代码的时候需要反映过来,与这里进行对照。

1.2 整体启动流程概述

本小节我们分析的是开源 SPL (u-boot 代码编译出来的) 的启动流程,该 SPL 镜像编译链接过程使用的是 arch/arm/cpu/armv8/u-boot-spl.lds 链接脚本。

整体启动流程如下:

```
arch/arm/cpu/armv8/start.S
```

```
_start:
```

```
    b    reset
    b    save_boot_params
    save_boot_params_ret
    bl   apply_core_errata
    bl   lowlevel_init      # arch/arm/cpu/armv8/start.S
        bl   s_init
    bl   _main              # arch/arm/lib/crt0_64.S
```

```
_main:      # arch/arm/lib/crt0_64.S
```

```
/* 为 C 语言代码运行设置栈地址 */
```

```
ldr x0, =(CONFIG_SPL_STACK) #CONFIG_SPL_STACK(include/configs/rk3568_common.h)
```

```
bic sp, x0, #0xf /* 16-byte alignment for ABI compliance */
mov x0, sp
bl board_init_f_alloc_reserve /* common/init/board_init.c
mov sp, x0
/* sp= CONFIG_SPL_STACK-CONFIG_SPL_SYS_MALLOC_F_LEN-sizeof(struct global_data) */
/* 栈向下生长 */

/* 设置 gd 的地址 */
bl board_init_f_init_reserve
/* 该函数指定了 gd 的基地址为 CONFIG_SPL_STACK、并指定了 malloc 的基地址 malloc_base
* 也就是在 gd 结构体之后, 紧跟 gd 结构体, 通过 gd->malloc_base 指定 */

/* 对 gd 进行填充 */
bl board_init_f_boot_flags /* arch/arm/mach-rockchip/board.c
/* 具体代码看 1.3 小节 */

bl board_init_f /* arch/arm/mach-rockchip/spl.c
bl spl_relocate_stack_gd /* 对于我们的 RK 平台 这里直接返回 0 没做什么事
b board_init_r
```

```
board_init_f: /* arch/arm/mach-rockchip/spl.c
rockchip_stimer_init
printascii("U-Boot SPL board init");
spl_early_init /* common/spl/spl.c
preloader_console_init
arch_cpu_init /* arch/arm/mach-rockchip/rk3568.c
rk_board_init_f /* 弱类型 空函数
```

```
board_init_r: /* common/spl/spl.c
spl_initr_dm #空函数
spl_set_bd
spl_board_init # arch/arm/mach-rockchip/spl.c
setup_led # 空函数
rk_spl_board_init # 空函数
spl_image.entry_point_bl32 = -1;
spl_image.entry_point_bl33 = CONFIG_SYS_TEXT_BASE;
board_boot_order(spl_boot_list); /* arch/arm/mach-rockchip/spl-boot-order.c
spl_next_stage(&spl_image); /* 弱类型 空函数
boot_from_devices /* common/spl/spl.c
spl_perform_fixups(&spl_image); /* arch/arm/mach-rockchip/spl.c
switch (spl_image.os) {
```

```

case IH_OS_ARM_TRUSTED_FIRMWARE:
    printf("Jumping to %s(0x%08lx) via ARM Trusted Firmware(0x%08lx)\n",
        spl_image.next_stage == SPL_NEXT_STAGE_UBOOT ? "U-Boot" :
        (spl_image.next_stage == SPL_NEXT_STAGE_KERNEL ? "Kernel" : "Unknown"),
        (ulong)spl_image.entry_point_bl33,
        (ulong)spl_image.entry_point);
    spl_invoke_atf(&spl_image);    //到这里就结束了 会在此函数中跳转到 BL31 执行
}

```

```

-----malloc_end(就等于CONFIG_SPL_STACK)
-->-->|
-->-->| ·+· CONFIG_SPL_SYS_MALLOC_F_LEN
-->-->|
-----malloc_base
-->-->|
-->-->| ·+· gd·size
-->-->|
-----栈顶（也是gd的起始地址）
·····| -->-->-->CONFIG_SPL_STACK·-
-->-->| ·-->-->-->CONFIG_SPL_SYS_MALLOC_F_LEN·-
-->-->| -->-->-->sizeof(struct·global_data)
-->-->|
-->-->|
-->-->|
-->-->V
-----DDRbase

```

1.3 board_init_f_boot_flags 函数

```

int board_init_f_boot_flags(void)
{
    int boot_flags = 0;

    /* pre-loader serial */
    #if defined(CONFIG_ROCKCHIP_PRELOADER_SERIAL) && \
        defined(CONFIG_ROCKCHIP_PRELOADER_ATAGS)
        struct tag *t;

        t = atags_get_tag(ATAG_SERIAL); // 可以获取到ATAG_SERIAL 这是TPL传递进来的 意味着串口是可以直接使用了
        if (t) { // 所以会进入到这个if分支 在这里对gd结构体中的某些变量进行初始化操作
            gd->serial.using_pre_serial = 1; // 所以意味着TPL已经对串口初始化过了
            gd->serial.enable = t->u.serial.enable; // 所以这里会获取到串口的一些信息
            gd->serial.baudrate = t->u.serial.baudrate;
            gd->serial.addr = t->u.serial.addr;
            gd->serial.id = t->u.serial.id;
            gd->baudrate = t->u.serial.baudrate;
            if (!t->u.serial.enable) // 这个条件不成立
                boot_flags |= GD_FLG_DISABLE_CONSOLE;
            debug("preloader: enable=%d, addr=0x%x, baudrate=%d, id=%d\n",
                t->u.serial.enable, (u32)t->u.serial.addr,
                t->u.serial.baudrate, t->u.serial.id);
        } else
    #endif
    {
        gd->baudrate = CONFIG_BAUDRATE;
        gd->serial.baudrate = CONFIG_BAUDRATE;
        gd->serial.addr = CONFIG_DEBUG_UART_BASE;
    }

    /* The highest priority to turn off (override) console */
    #if defined(CONFIG_DISABLE_CONSOLE) // 我们没有定义这个
        boot_flags |= GD_FLG_DISABLE_CONSOLE;
    #endif

    return boot_flags; // 所以这里返回就是0
} ? end board_init_f_boot_flags ?

```

1.4 board_init_f 函数

arch/arm/mach-rockchip/spl.c


```

void board_init_f(ulong dummy)
{
#ifdef CONFIG_SPL_FRAMEWORK           //对于SPL定义了这个
    int ret;
    #if !defined(CONFIG_SUPPORT_TPL)
        struct udevice *dev;
    #endif
#endif

    rockchip_stimer_init();           //对定时器进行一个简单地初始化操作
#define EARLY_UART
    #if defined(EARLY_UART) && defined(CONFIG_DEBUG_UART)
        /*
         * Debug UART can be used from here if required:
         */
        * debug_uart_init();
        * printch('a');
        * printhex8(0x1234);
        * printascii("string");
        */
        if (!gd->serial.using_pre_serial && //这些条件不成立 因为using_pre_serial=1
            ! (gd->flags & GD_FLG_DISABLE_CONSOLE))
            debug_uart_init();
        printascii("U- Boot SPL board init"); //打印这句话 这是SPL打印的第一句字符串
    #endif
    gd->sys_start_tick = get_ticks0; //获取一个开始时间 估计是用于后面打印SPL耗时而设计的
    #ifdef CONFIG_SPL_FRAMEWORK
        ret = spl_early_init();
        if (ret) {

            printf("spl_early_init() failed: %d\n", ret);
            hang0;
        }
    #if !defined(CONFIG_SUPPORT_TPL)
        debug("\nspl:init dram\n");
        ret = uclass_get_device(UCLASS_RAM, 0, &dev);
        if (ret) {
            printf("DRAM init failed: %d\n", ret);
            return;
        }
    #endif
    #endif
    preloader_console_init();
    #else
        /* Some SoCs like rk3036 does not use any frame work */
        sdram_init();
    #endif

    arch_cpu_init(); // 这里涉及到RK平台 相关的一些底层初始化操作 不用去理会
    rk_board_init_f(); //对RK3568来说是 弱类型 空函数
    #if CONFIG_IS_ENABLED(ROCKCHIP_BACK_TO_BROM) && !defined(CONFIG_SPL_BOARD_INIT)
        back_to_bootrom(BROM_BOOT_NEXTSTAGE); //对于SPL 不会执行这个函数
    #endif

} ? end board_init_f ?

```

board_init_f 函数中的重点就是在于 spl_early_init 函数, 该函数中会对设备树进行解析、根据设备树配置信息加载对应的驱动程序。

preloader_console_init 函数, 对于 SPL RK3568 平台来说, 如下:


```
void preloader_console_init(void)
{
    gd->baudrate = CONFIG_BAUDRATE;    //波特率

    serial_init();    /* serial communications setup */ //对于我们来说 这个空函数

    gd->have_console = 1;    //表示我们有终端了

    puts("\nU-Boot SPL " PLAIN_VERSION " (" U_BOOT_DATE " - " \
        U_BOOT_TIME ")\n");    //打印信息 包括时间和日期
#ifdef CONFIG_SPL_DISPLAY_PRINT
    spl_display_print();    //这个不会执行 因为CONFIG_SPL_DISPLAY_PRINT没定义
#endif
}
```

puts 打印的语句其实就是 SPL 程序 运行打印的第二句字符串, 如下所示:

```
U-Boot SPL board init
U-Boot SPL 2017.09 #alientek (Oct 15 2021 - 10:31:29)
unknown raw ID pHN
unrecognized JEDEC id bytes: 00, 00, 00
Trying to boot from MMC2
MMC error: The cmd index is 1, ret is -110
Card did not respond to voltage select!
mmc_init: -95, time 9
spl: mmc init failed with error: -95
Trying to boot from MMC1
SPL: A/B-slot: _a, successful: 0, tries-remain: 7
## Verified-boot: 0
## Checking atf-1 0x00040000 ... sha256+ OK
## Checking uboot 0x00a00000 ... sha256+ OK
```

1.4.1 spl_early_init 函数

Spl.c(common\spl)

```
int spl_early_init(void)
{
    int ret;

    ret = spl_common_init(true);    //调用spl_common_init函数
    if (ret)
        return ret;
    gd->flags |= GD_FLG_SPL_EARLY_INIT;    //给gd的flags标志加上GD_FLG_SPL_EARLY_INIT
    //表示早期 SPL 初始化完成
    spl_setup_relocate();    //最后调用spl_setup_relocate函数 对于SPL来说 这是空函数

    return 0;
}
```

看看 spl_common_init 函数:

```

static int spl_common_init(bool setup_malloc)
{
    int ret;

    debug("spl_early_init()\n");

    #if CONFIG_VAL(SYS_MALLOC_F_LEN)
        if (setup_malloc) {
    #ifdef CONFIG_MALLOC_F_ADDR // 我们没定义这个宏 如果定义了这个宏 则使用该宏对应的值 作为malloc基地址
        gd->malloc_base = CONFIG_MALLOC_F_ADDR;
    #endif
        gd->malloc_limit = CONFIG_VAL(SYS_MALLOC_F_LEN); //malloc地址空间的最大值吧 应该是
        gd->malloc_ptr = 0; //这个应该指的是当前的 malloc 所指向的位置偏移量
        }
    #endif

    /*
     * setup D- cache as early as possible after malloc setup
     * I- cache has been setup at early assembly code by default.
     */
    #ifndef CONFIG_SPL_SYS_DCACHE_OFF
        ret = spl_dcachel_enable(); //打开DCache
        if (ret) {
            debug("spl_dcachel_enable() return error %d\n", ret);
            return ret;
        }
    #endif
    ret = bootstage_init(true); //对于SPL来说 我们没有使能bootstage
    if (ret) { //所以这是一个返回0 的空函数
        debug("%s: Failed to set up bootstage: ret=%d\n", __func__,
            ret);

        return ret;
    }
    bootstage_mark_name(BOOTSTAGE_ID_START_SPL, "spl");//空函数
    if (CONFIG_IS_ENABLED(OF_CONTROL) && ! CONFIG_IS_ENABLED(OF_PLATDATA)) {
        ret = fdtdec_setup(); //解析设备树 对于SPL来说, 也有设备树
        if (ret) {
            debug("fdtdec_setup() returned error %d\n", ret);
            return ret;
        }
    }
    if (CONFIG_IS_ENABLED(DM)) {
        bootstage_start(BOOTSTATE_ID_ACCUM_DM_SPL, "dm_spl");
        /* With CONFIG_SPL_OF_PLATDATA, bring in all devices */
        ret = dm_init_and_scan(! CONFIG_IS_ENABLED(OF_PLATDATA)); //扫描、加载驱动外设驱动
        bootstage_accum(BOOTSTATE_ID_ACCUM_DM_SPL);
        if (ret) {
            debug("dm_init_and_scan() returned error %d\n", ret);
            return ret;
        }
    }

    return 0;
} ? end spl_common_init ?

```

调用 `dm_init_and_scan` 函数扫描设备树, 加载对应的驱动, 对于 SPL 来说, 这里只能加载设备节点中定义了 "u-boot,dm-spl" 或 "u-boot,dm-pre-reloc" 属性的设备; 事实上, SPL 的设备树在编译时使用的也是 u-boot 源码中提供的设备树文件, 譬如对于 RK3568 平台来说就是 `arch/arm/dts/rk3568-evb.dts`, 但是在编译 SPL 的设备树镜像时, 会过滤出定义了 "u-boot,dm-spl" 或 "u-boot,dm-pre-reloc" 属性的节点, 其余的节点都会被剔除

(特殊节点除外, 譬如 chosen、aliases), 所以会使得, SPL 的 dtb 文件相比于原始的 u-boot 设备树镜像文件要小。

在 RK 提供的 u-boot 中, 采用的是分离的方式遍历设备树, 我们一般编译 u-boot 采用的是嵌入的方式, 也就是说设备树镜像会嵌入到 u-boot 镜像中; 而分离(独立)方式, 则是和内核一样, 设备树镜像不会嵌入到 u-boot 镜像中, 而是单独编译出来提供。

1.5 board_init_r 函数

common/spl/spl.c

```
void board_init_r(gd_t *dummy1, ulong dummy2)
{
    u32 spl_boot_list[] = {
        BOOT_DEVICE_NONE,
        BOOT_DEVICE_NONE,
        BOOT_DEVICE_NONE,
        BOOT_DEVICE_NONE,
        BOOT_DEVICE_NONE,
    };
    struct spl_image_info spl_image; // 这个结构体比较重要 里边记录了BL1 BL2 BL3的其实位置 当然会在后面的过程进行解析、填充
    debug(">>spl:board_init_r()\n");
    spl_initr_dm(); // 对我们来说 是空函数 直接返回0
    spl_set_bd(); // 这里其实就是分配了 gd->bd 数据块 因为前面过程中没有去设置bd数据块
#ifdef CONFIG_SPL_OS_BOOT // 没定义这个宏
    dram_init_banksize();
#endif
#ifdef CONFIG_SYS_SPL_MALLOC_START // 没定义这个宏
    mem_malloc_init(CONFIG_SYS_SPL_MALLOC_START,
        CONFIG_SYS_SPL_MALLOC_SIZE);
    gd->flags |= GD_FLG_FULL_MALLOC_INIT;
#endif
    if (! (gd->flags & GD_FLG_SPL_INIT)) { // 因为GD_FLG_SPL_INIT在前面已经设置了 所以这个if条件不会成立
        if (spl_init()) // 也就是说前面已经完成了SPL初始化
            hang();
    }
}
```

```

#ifndef CONFIG_PPC) && ! defined(CONFIG_ARCH_MX6)
/*
 * timer_init() does not exist on PPC systems. The timer is initialized
 * and enabled (decrementer) in interrupt_init() here.
 */
timer_init(); //RK没实现 使用默认的弱类型空函数 直接返回0
#endif

#if CONFIG_IS_ENABLED(BOARD_INIT) //CONFIG_SPL_BOARD_INIT有定义 所以这里会执行
    spl_board_init(); //对于RK3568来说 是空函数
#endif

memset(&spl_image, '\0', sizeof(spl_image)); //对spl_image变量清零操作

#if CONFIG_IS_ENABLED(ATF) //定义了这个宏 CONFIG_SPL_ATF
/*
 * BL32 ep is optional, initial it as an invalid value.
 * BL33 ep is mandatory, but initial it as a default value is better.
 */
spl_image.entry_point_bl32 = - 1;
spl_image.entry_point_bl33 = CONFIG_SYS_TEXT_BASE; //BL33 对于我们这个平台来说 就是u-boot代码
//事实上, CONFIG_SYS_TEXT_BASE就是u-boot的链接地址
//但是后面会通过FIT解析 获取到它们的入口地址 这里设置与否都无关
#endif

#if CONFIG_IS_ENABLED(OPTEE) //采用ATF 不使用OPTEE
/* default address */
spl_image.entry_point_os = CONFIG_SYS_TEXT_BASE;
#endif

#ifdef CONFIG_SYS_SPL_ARGS_ADDR //RK平台 没定义该宏
    spl_image.arg = (void *)CONFIG_SYS_SPL_ARGS_ADDR;
#endif
spl_image.boot_device = BOOT_DEVICE_NONE;
board_boot_order(spl_boot_list); //这个函数会通过设备树的找出所有定义的可用于启动的设备 以ID编号表示
spl_next_stage(&spl_image); //完成置位spl->next_stage = SPL_NEXT_STAGE_UBOOT
if (boot_from_devices(&spl_image, spl_boot_list, //该函数会从启动设备树中 加载镜像 也就是uboot.img
    ARRAY_SIZE(spl_boot_list))) { //uboot.img包括了 bl31 bl32 bl33
    puts("SPL: failed to boot from all boot devices\n"); //会将它们加载到内存中
    hang();
}

spl_perform_fixups(&spl_image); //通过RK平台实现的ATAGS机制 进行传参, 这里主要是传递了 启动设备这个参数
//估计是用于告知后续的镜像 当前板子的启动设备 注意ATAGS传参机制是RK实现的

#ifdef CONFIG_CPU_V7M
    spl_image.entry_point |= 0x1;
#endif
switch (spl_image.os) {
    case IH_OS_U_BOOT:
        debug("Jumping to U-Boot\n");
        break;
    #if CONFIG_IS_ENABLED(ATF)
        case IH_OS_ARM_TRUSTED_FIRMWARE: //我们是进入这个分支
            printf("Jumping to %s(0x%08lx) via ARM Trusted Firmware(0x%08lx)\n",
                spl_image.next_stage == SPL_NEXT_STAGE_UBOOT ? "U-Boot" :
                (spl_image.next_stage == SPL_NEXT_STAGE_KERNEL ? "Kernel" : "Unknown"),
                (ulong)spl_image.entry_point_bl33, //这里会打印出一些信息 在SPL运行打印信息中是可以找到的
                (ulong)spl_image.entry_point);
            spl_invoke_atf(&spl_image); //最终通过该函数 跳转到 BL1镜像执行 那么会由BL1去引导后续的BL2 u-boot之类的
            break;
    #endif
}

```

这里面的重点函数在于 board_boot_order、boot_from_devices、spl_invoke_atf, 大概就是:

board_boot_order 函数扫描出可用于启动的设备;

boot_from_devices 从启动介质中加载镜像到内存;

spl_invoke_atf 就是启动镜像。

当然里面涉及到很多的细节。

spl_perform_fixups(&spl_image);这个函数主要做的事就是传参,通过 RK 的 ATAGS 传参机制实现传参,因为通过 boot_from_devices 函数之后,他就知道了是从那个设备加载的镜像,那么也就是意味着该设备就

是启动设备,所以它会把这个信息告知后面的镜像,通过这个 ATAGS 传参机制,这个机制其实很简单,会在另一份文档中对其进行分析。

1.5.1 board_boot_order 函数

spl-boot-order.c (arch/arm/mach-rockchip)

```
void board_boot_order(u32 *spl_boot_list)
{
    const void *blob = gd->fdt_blob; // 首先从gd中获取到SPL设备树镜像的加载地址
    int chosen_node = fdt_path_offset(blob, "/chosen"); // 找根节点下的chosen节点
    int idx = 0;
    int elem;
    int boot_device;
    int node;
    const char *conf;

    if (chosen_node < 0) { // 如果找不到/chosen节点 在我们RK平台设备树中肯定是定义了这个节点的
        debug("%s: /chosen not found, using spl_boot_device()\n",
            __func__);
        spl_boot_list[0] = spl_boot_device();
        return;
    }

    for (elem = 0;
        (conf = fdt_stringlist_get(blob, chosen_node, // 遍历"u-boot,spl-boot-order"属性中的内容
            "u-boot,spl-boot-order", elem, NULL)); // "u-boot,spl-boot-order"其实就是一个数组
        elem++) { // 它记录了一些存储设备,表示这些存储设备可用于启动系统
        const char *alias;

        /* Handle the case of 'same device the SPL was loaded from' */
        if (strcmp(conf, "same-as-spl", 11) == 0) { // 如果等于"same-as-spl"
            conf = board_spl_was_booted_from(); // 这个不知道是啥意思
            if (!conf) // 猜测应该是跟启动SPL的设备 同一个设备
                continue; // 也就是说启动SPL是哪个设备 就是这个设备
            // 这个应该是由BootROM代码写进去的 BootROM代码会将加载Miniloader.bin的设备作为启动设备
            // 然后将这个启动设备信息写入到一个寄存器中
        }
        /* First check if the list element is an alias */
        alias = fdt_get_alias(blob, conf); // 校验这个名称是不是别名
        if (alias)
            conf = alias; // 如果是别名的话 需要将conf替换为 实际的节点路径

        /* Try to resolve the config item (or alias) as a path */
        node = fdt_path_offset(blob, conf); // 然后去找这个节点
        if (node < 0) {
            debug("%s: could not find %s in FDT", __func__, conf);
            continue;
        }

        /* Try to map this back onto SPL boot devices */
        boot_device = spl_node_to_boot_device(node); // 这个函数功能 其实是做转换的
        if (boot_device < 0) { // 它会将上面从设备树中识别到的存储设备 转换为一个 ID值
            debug("%s: could not map node @%x to a boot-device\n",
                __func__, node);
            continue;
        }

        spl_boot_list[idx++] = boot_device; // 将这个ID值 存放在这个数组中
    } // ? end for elem=0;(conf=fdt_stri... ?

    /* If we had no matches, fall back to spl_boot_device */
    if (idx == 0) // 如果这个idx等于0 表示上面没有找到 任何启动设备 然后才会执行下面的这个函数
        spl_boot_list[0] = spl_boot_device(); // 也就使用一个默认的设备作为启动设备 这个设备是MMC1
    } // ? end board_boot_order ?
```

重点就是 `spl_node_to_boot_device` 函数, 做转换的, 它会转成一个 ID 值, 看下就明白了:

```
enum {
    BOOT_DEVICE_RAM,
    BOOT_DEVICE_MMC1,
    BOOT_DEVICE_MMC2,
    BOOT_DEVICE_MMC2_2,
    BOOT_DEVICE_NAND,
    BOOT_DEVICE_ONENAND,
    BOOT_DEVICE_NOR,
    BOOT_DEVICE_UART,
    BOOT_DEVICE_SPI,
    BOOT_DEVICE_USB,
    BOOT_DEVICE_SATA,
    BOOT_DEVICE_I2C,
    BOOT_DEVICE_BOARD,
    BOOT_DEVICE_DFU,
    BOOT_DEVICE_XIP,
    BOOT_DEVICE_BOOTROM,
    BOOT_DEVICE_MTD_BLK_NAND,
    BOOT_DEVICE_MTD_BLK_SPI_NAND,
    BOOT_DEVICE_MTD_BLK_SPI_NOR,
    BOOT_DEVICE_RKNAND,
    BOOT_DEVICE_NONE
};
```

它怎么转的呢? 这个其实很简单, 得益于 u-boot DM 框架的一个实现, 具体的实现方法就不介绍了! 所以经过 `board_boot_order` 函数之后, `spl_boot_list` 数组中就存放了一些可用于启动的设备 ID。

1.5.2 boot_from_devices 函数

spl.c (common\spl)

```
static int boot_from_devices(struct spl_image_info *spl_image,
                             u32 spl_boot_list[], int count)
{
    int i;

    for (i = 0; i < count && spl_boot_list[i] != BOOT_DEVICE_NONE; i++) { //遍历spl_boot_list数组
        struct spl_image_loader *loader; //这个结构体 怎么说呢 这个结构体记录了这些启动设备(存储介质)
        //加载镜像的一个方法 也就是提供了一个读取镜像、加载镜像到内存的一个方法
        loader = spl_ll_find_loader(spl_boot_list[i]); //会通过spl_ll_find_loader找到 对应启动设备的spl_image_loader对象
        #if defined(CONFIG_SPL_SERIAL_SUPPORT) && defined(CONFIG_SPL_LIBCOMMON_SUPPORT)
            if (loader)
                printf("Trying to boot from %s\n", loader->name); //它找到之后 会打印出这句字符串 在终端可以看到的
            else
                puts("SPL: Unsupported Boot Device! \n");
        #endif
        if (loader && ! spl_load_image(spl_image, loader)) { //最终调用spl_load_image函数 去加载镜像到内存
            spl_image->boot_device = spl_boot_list[i]; //如果成功了 会将这个启动设备记录到 spl_image->boot_device中
            return 0; //后续可能会使用这个
        }
    }

    return - ENODEV;
} // end boot_from_devices ?
```

struct `spl_image_loader` 结构体描述了启动设备的加载镜像的方法, 如下:


```

struct spl_image_loader {
#ifdef CONFIG_SPL_LIBCOMMON_SUPPORT
    const char *name;
#endif
    uint boot_device;
    /**
     * load_image() - Load an SPL image
     *
     * @spl_image: place to put image information
     * @bootdev: describes the boot device to load from
     */
    int (*load_image)(struct spl_image_info *spl_image,
                      struct spl_boot_device *bootdev);
};

```

但是需要通过 `spl_ll_find_loader` 函数找到它的加载方法, 怎么找到呢? 两个条件: 一个是启动设备的 ID、另一个需要 `SPL_LOAD_IMAGE_METHOD` 宏来实现。在 `common/spl` 目录下有很多 `spl_xxx.c` 的文件, 这个 `xxx` 是某一个存储介质的名称, 譬如 `spi`、`mmc`、`nand` 等, 在这些文件中会通过 `SPL_LOAD_IMAGE_METHOD` 宏注册一个加载方法, 如下所示:

```

SPL_LOAD_IMAGE_METHOD("MMC1", 0, BOOT_DEVICE_MMC1, spl_mmc_load_image);
SPL_LOAD_IMAGE_METHOD("MMC2", 0, BOOT_DEVICE_MMC2, spl_mmc_load_image);
SPL_LOAD_IMAGE_METHOD("MMC2_2", 0, BOOT_DEVICE_MMC2_2, spl_mmc_load_image);

```

注册的原理其实就是将加载方法以及用于识别该设备的 ID 封装成一个数据结构, 然后编译链接的时候放置到一个特定的段中, 所以 `spl_ll_find_loader` 函数就是去遍历这个段中的数据结构, 根据 ID 找到对应加载函数即可!

所以说, 如果要分析 SPL 是如何加载后续的 `uboot.img` 镜像的, 那就要分析这个加载函数, 譬如对于 `mmc` 设备来说, 就要分析 `spl_mmc_load_image` 函数内部的实现细节!

1.5.3 spl_invoke_atf 函数

`spl_atf.c` (`common\spl`)


```

void spl_invoke_atf(struct spl_image_info *spl_image)
{
    uintptr_t bl32_entry, bl33_entry;
    void *blob = spl_image->fdt_addr; //spl_image->fdt_addr记录的是 uboot的设备树镜像 不是spl的镜像 要注意
    uintptr_t platform_param = (uintptr_t)blob;
    int node;

    /*
     * Find the OP-TEE binary (in /fit-images) load address or
     * entry point (if different) and pass it as the BL3-2 entry
     * point, this is optional.
     * This will need to be extended to support Falcon mode.
     */
    node = spl_fit_images_find(blob, IH_OS_OP_TEE);
    if (node >= 0)
        bl32_entry = spl_fit_images_get_entry(blob, node); // 设置BL32的加载地址(optee)
    else
        bl32_entry = spl_image->entry_point_bl32; /* optional */

    /*
     * Find the U-Boot binary (in /fit-images) load address or
     * entry point (if different) and pass it as the BL3-3 entry
     * point.
     * This will need to be extended to support Falcon mode.
     */
    node = spl_fit_images_find(blob, IH_OS_U_BOOT);
    if (node >= 0)
        bl33_entry = spl_fit_images_get_entry(blob, node); // 设置BL33的加载地址(u-boot)
    else
        bl33_entry = spl_image->entry_point_bl33;

    /*
     * If ATF_NO_PLATFORM_PARAM is set, we override the platform
     * parameter and always pass 0. This is a workaround for
     * older ATF versions that have insufficiently robust (or
     * overzealous) argument validation.
     */
    if (CONFIG_IS_ENABLED(ATF_NO_PLATFORM_PARAM))
        platform_param = 0;

    /* do cleanup */
    spl_cleanup_before_jump(spl_image);

    /*
     * We don't provide a BL3-2 entry yet, but this will be possible
     * using similar logic.
     */
    bl31_entry(spl_image->entry_point, bl32_entry, // 执行跳转, spl_image->entry_point
               bl33_entry, platform_param); // spl_image->entry_point记录的是BL31的入口地址
    // 所以由此可知, SPL之后是BL31运行
} ? end spl_invoke_atf ?

```

spl_fit_images_find 函数会在 u-boot 的设备树中查找, "/fit-images"节点, 然后遍历这个节点下的所有子节点, 判断每一个子节点的"os"属性, 譬如 spl_fit_images_find(blob, IH_OS_OP_TEE)就是判断这些子节点中, 它们的 os 属性是否等于 IH_OS_OP_TEE(这是一个 enum 枚举常量), 如果确实相等, 那么就会返回该子节点, 所以自然返回的 node 就不会等于 0。

事实上, 在我们的 u-boot 设备树中, 并没有"/fit-images"节点, 因为这个节点是由 SPL 去填充进去的, SPL 在加载镜像的过程会去创建出这个节点, 这个后面再去分析。

当执行 bl31_entry 函数的时候, 就会跳转到 spl_image->entry_point 地址, 也就是 BL31 的入口地址去执行, 并把 bl32 和 bl33 的入口地址也一并传递进去, 因为 bl31 代码运行完之后, 便会启动 bl32 → bl33 这样, 所以必须得知道他们的入口地址, 其次还会将 u-boot 的设备树镜像的加载地址 platform_param 也一并传递进去, 估计后续会将其传递给 u-boot。

spl_cleanup_before_jump 函数中会做一些最后的清理工作, 譬如处理中断、cache、等:

```
void spl_cleanup_before_jump(struct spl_image_info *spl_image)
{
    ulong us;

    spl_board_prepare_for_jump(spl_image);

    disable_interrupts(); // 禁止中断

    /*
     * Turn off I- cache and invalidate it
     */
    icache_disable(); // 关闭icache
    invalidate_icache_all();

    /*
     * Turn off D- cache
     * dcache_disable() in turn flushes the d- cache and disables MMU
     */
    dcache_disable(); // 关闭dcache
    invalidate_dcache_all();

    dsb();
    isb();

    us = (get_ticks() - gd->sys_start_tick) / 24UL;
    printf("Total: %ld.%ld ms\n\n", us / 1000, us % 1000); // 最后打印TPL运行所消耗的总时间
} // end spl_cleanup_before_jump ?
```

1.6 spl_mmc_load_image 函数

以 mmc 设备加载位置, 来看看 spl_mmc_load_image 加载函数是如何从外接的存储设备中读取镜像到内存中。

spl_mmc.c (common\spl)

```
spl_mmc_load_image
    mmc_load_image_raw_partition
        mmc_load_image_raw_sector
            spl_load_simple_fit
                spl_internal_load_simple_fit
                    {
                        .....
                        spl_load_fit_image
                        .....
                    }
```

```

int spl_mmc_load_image(struct spl_image_info *spl_image, //从board_init_r函数传递进来的
                      struct spl_boot_device *bootdev)
{
    struct mmc *mmc = NULL;
    u32 boot_mode;
    int err = 0;
    __maybe_unused int part;

    err = spl_mmc_find_device(&mmc, bootdev->boot_device); //首先通过boot_device(也就是ID)找到对应的mmc设备
    if (err) //找到之后 通知mmc指向它
        return err;

    err = mmc_init(mmc); //执行mmc初始化操作
    if (err) {
#ifdef CONFIG_SPL_LIBCOMMON_SUPPORT
        printf("spl: mmc init failed with error: %d\n", err);
#endif
        return err;
    }

#ifdef CONFIG_SPL_LOAD_RKFW //我们没有定义这个宏
    struct spl_load_info load;

    load.dev = mmc_get_blk_desc(mmc);
    load.priv = NULL;
    load.filename = NULL;
    load.bl_len = mmc->read_bl_len;
    load.read = h_spl_load_read;

    err = spl_load_rkfw_image(spl_image, &load);
    if (!err || err != -EAGAIN)
        return err;
#endif
    boot_mode = spl_boot_mode(bootdev->boot_device); //获取存储设备的启动模式
    err = -EINVAL; //其实就是说镜像是以文件系统的方式存储的还是直接基于扇区(raw原始方式)
    switch (boot_mode) { //对于RK平台来说 就是RAW
        case MMCSD_MODE_EMMCBOOT:
            /*
             * We need to check what the partition is configured to.
             * 1 and 2 match up to boot0 / boot1 and 7 is user data
             * which is the first physical partition (0).
             */
            part = (mmc->part_config >> 3) & PART_ACCESS_MASK;

            if (part == 7)
                part = 0;

            if (CONFIG_IS_ENABLED(MMC_TINY))
                err = mmc_switch_part(mmc, part);
            else
                err = blk_dselect_hwpart(mmc_get_blk_desc(mmc), part);

            if (err) {
#ifdef CONFIG_SPL_LIBCOMMON_SUPPORT
                puts("spl: mmc partition switch failed\n");
#endif
                return err;
            }
            /* Fall through */
        case MMCSD_MODE_RAW: //所以会进入到这里case分支
            debug("spl: mmc boot mode: raw\n");
    }
}

```

```
if (! spl_start_uboot()) { //这个函数是空函数 直接返回1 所以这个if不会成立
    err = mmc_load_image_raw_os(spl_image, mmc);
    if (! err)
        return err;
}
#ifdef CONFIG_SYS_MMCSL_RAW_MODE_U_BOOT_USE_PARTITION //这个宏我们定义了
    err = mmc_load_image_raw_partition(spl_image, mmc, //加载镜像
        CONFIG_SYS_MMCSL_RAW_MODE_U_BOOT_PARTITION_NAME,
        CONFIG_SYS_MMCSL_RAW_MODE_U_BOOT_PARTITION);
    if (! err)
        return err; //如果加载成功 则这里会直接退出
#endif
#ifdef CONFIG_SYS_MMCSL_RAW_MODE_U_BOOT_USE_SECTOR //这个宏没定义
    err = mmc_load_image_raw_sector(spl_image, mmc,
        CONFIG_SYS_MMCSL_RAW_MODE_U_BOOT_SECTOR);
    if (! err)
        return err;
#endif
```

spl_mmc_load_image 函数的重点是调用 mmc_load_image_raw_partition 函数从 mmc 中加载进行, 随同传递了两个参数进去, CONFIG_SYS_MMCSL_RAW_MODE_U_BOOT_PARTITION_NAME 指的是分区名字, 表示我们要从哪个分区加载镜像, 这个在 u-boot 中是可以进行配置的, RK 平台默认将其设置为"uboot", 表示我们将要从 uboot 分区中加载 uboot.img 镜像。

使用 RK 的烧写工具时, 我们会提供一个 parameter.txt 分区表文件, 当中就定义了各个分区, 所以这些分区名字不可随意更改, 因为在源码中已经使用了这些名字, 如果你要改, 除非将源码也一并修改了。

再看看 CONFIG_SYS_MMCSL_RAW_MODE_U_BOOT_PARTITION 参数, 这其实是分区的编号, 也就是说如果你提供的分区表的名字不对, 或者根本没有这个分区, 那么它就会去加载该编号对应的分区 (譬如 0、1、2、3、4, 这个下标应该是从 0 开始的)。

1.6.1 mmc_load_image_raw_partition 函数

```

#ifdef CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_USE_PARTITION
static int mmc_load_image_raw_partition(struct spl_image_info *spl_image,
                                       struct mmc *mmc,
                                       const char *partition_name,
                                       int partition)
{
    disk_partition_t info;
    int err;

#ifdef CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_USE_PARTITION_TYPE // 没定义该宏
    int type_part;
    /* Only support MBR so DOS_ENTRY_NUMBERS */
    for (type_part = 1; type_part <= DOS_ENTRY_NUMBERS; type_part++) {
        err = part_get_info(mmc_get_blk_desc(mmc), type_part, &info);
        if (err)
            continue;
        if (info.sys_ind ==
            CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_PARTITION_TYPE) {
            partition = type_part;
            break;
        }
    }
#endif
    if (strcmp(partition_name, "")) // 如果传递进来的分区名不等于空
        err = part_get_info_by_name(mmc_get_blk_desc(mmc), // 那么就会通过分区名的方式
                                    partition_name, &info); // 找到该分区的信息 通过info返回出来
    else // 所以info对象中就记录了分区的信息 譬如起始block 分区大小
        err = part_get_info(mmc_get_blk_desc(mmc), partition, &info);
    if (err < 0) { // 如果传递进来的分区名是空的 则会使用分区编号
#ifdef CONFIG_SPL_LIBCOMMON_SUPPORT
        puts("spl: partition error\n"); // 如果找不到对应的分区 就会打印这句话
#endif
    }

    return -1;
}

#ifdef CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_USE_SECTOR
return mmc_load_image_raw_sector(spl_image, mmc,
                                info.start + CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR);
#else
return mmc_load_image_raw_sector(spl_image, mmc, info.start); // 调用这个
#endif
} ? end mmc_load_image_raw_partition ?
#endif

```

当调用 mmc_load_image_raw_sector 函数时候, 会将分区的其实地址 (也就是 block 块, 其实块) 传递进去, 通过 info.start 参数。

1.6.2 A/B 分区的处理

在 part_get_info_by_name 函数还有一个处理就是关于 A/B 分区的问题, 默认我们的 SPL 支持 A/B 分区, 此时需要使用 misc 分区, 我们分析一下:

```

int part_get_info_by_name(struct blk_desc *dev_desc, const char *name,
                          disk_partition_t *info)
{
    struct part_driver *part_drv;
    char name_slot[32] = {0};
    int none_slot_try = 1;
    int ret, i;

    part_drv = part_driver_lookup_type(dev_desc);
    if (!part_drv)
        return -1;
    #if defined(CONFIG_ANDROID_AB) || defined(CONFIG_SPL_AB) //我们定义了CONFIG_SPL_AB
        char *name_suffix = (char *)name + strlen(name) - 2; //将name_suffix定位到name分区名的后两个字符
        //也就是它这里说的后缀
        /* Fix can not find partition with suffix "_a" & "_b". If with them, clear */
        if (!memcmp(name_suffix, "_a", strlen("_a")) || !memcmp(name_suffix, "_b", strlen("_b")))
            memset(name_suffix, 0, 2); //这段代码主要是处理_a或_b后缀的问题
    #endif //如果传进来的分区名带有_a或_b后缀 则需要把这个后缀去掉
    #if defined(CONFIG_ANDROID_AB) && !defined(CONFIG_SPL_BUILD) //对于SPL来说这个不成立
        /* 1. Query partition with A/B slot suffix */
        if (rk_avb_append_part_slot(name, name_slot))
            return -1;
    #elif defined(CONFIG_SPL_AB) && defined(CONFIG_SPL_BUILD) //这个成立
        if (spl_ab_append_part_slot(dev_desc, name, name_slot)) //会执行spl_ab_append_part_slot函数
            return -1; //该函数会处理A/B分区的问题 确定最终是哪个分区名 最终的名字通过name_slot返回出来
    #else
        strcpy(name_slot, name);
    #endif
    retry:
    debug("## Query partition(%d): %s\n", none_slot_try, name_slot);
    for (i = 1; i < part_drv->max_entries; i++) {
        ret = part_drv->get_info(dev_desc, i, info); //获取分区的信息存放在info对象中
        if (ret != 0) {
            /* no more entries in table */
            break;
        }
        if (strcmp(name_slot, (const char *)info->name) == 0) { //对比实际分区名字和name_slot
            /* matched */
            return i; //如果匹配成功直接返回 表示已经找到了这个分区
        }
    }

    /* 2. Query partition without A/B slot suffix if above failed */
    if (none_slot_try) { //如果上面 没有匹配成功 表示设备的分区表上压根就没有A/B槽的分区
        none_slot_try = 0; //也就是说没有_a或_b后缀的分区
        strcpy(name_slot, name); //那么它会直接使用上面处理的name 再去进行匹配
        goto ↑retry;
    }

    return -1;
} // end part_get_info_by_name ?

```

其中 spl_ab_append_part_slot 函数会去处理 A/B 分区的问题，如何处理，代码我们就不看了，直接说明一下思路：

首先 spl_ab_append_part_slot 函数返回的分区名 name_slot 可能取值是这样的（假设 name="uboot"，如果传入的 misc，传出的也是 misc）：

```

uboot_a
uboot_b
uboot

```


这三种情况, 只有当不存在 misc 分区时, 传出的 name_slot 才等于 uboot。

所以说, 只要存在 misc 分区, 那么传出的名字就一定是 uboot_a 或 uboot_b 这两者之一。那如何来确定到底是哪一个呢? 其实它会读取 misc 分区中一个数据结构, 通过这个数据结构记录的信息来决定, 这个数据结构就是 AvbABData (从偏移 misc 分区起始地址的 2KB 位置读取), 该结构体中有一个 AvbABSlotData 数组, 关于这两个结构体描述如下:

```
typedef struct AvbABData {
    /* Magic number used for identification - see AVB_AB_MAGIC. */
    uint8_t magic[AVB_AB_MAGIC_LEN];

    /* Version of on-disk struct - see AVB_AB_[MAJOR,MINOR]_VERSION. */
    uint8_t version_major;
    uint8_t version_minor;

    /* Padding to ensure |slots| field start eight bytes in. */
    uint8_t reserved1[2];

    /* Per-slot metadata. */
    AvbABSlotData slots[2];
}

/* Reserved for future use. */
uint8_t last_boot;
uint8_t reserved2[11];

/* CRC32 of all 28 bytes preceding this field. */
uint32_t crc32;
} ? end AvbABData ? AVB_ATTR_PACKED AvbABData;

typedef struct AvbABSlotData {
    /* Slot priority. Valid values range from 0 to AVB_AB_MAX_PRIORITY,
     * both inclusive with 1 being the lowest and AVB_AB_MAX_PRIORITY
     * being the highest. The special value 0 is used to indicate the
     * slot is unbootable.
     */
    uint8_t priority;

    /* Number of times left attempting to boot this slot ranging from 0
     * to AVB_AB_MAX_TRIES_REMAINING.
     */
    uint8_t tries_remaining;

    /* Non-zero if this slot has booted successfully, 0 otherwise. */
    uint8_t successful_boot;

    /* Reserved for future use. */
    uint8_t reserved[1];
} AVB_ATTR_PACKED AvbABSlotData;
```

AvbABData:

参数	含义
priority	标志 slot 优先级, 0 为不可启动, 15 为最高优先级
tries_remaining	尝试启动次数, 设置为 7 次
successful_boot	系统启动成功后会配置该参数, 1: 该 slot 成功启动, 0: 该 slot 未成功启动
is_update	标记该 slot 的升级状态, 1: 该 slot 正在升级, 0: 该 slot 未升级或升级成功

AvbABSlotData:

参数	含义
magic	结构体头部信息: \0AB0
version_major	主版本信息
version_minor	次版本信息
slots	slot 引导信息, 参见 AvbABData
last_boot	上一次成功启动的 slot, 0: slot A 上次成功启动, 1: slot B 上次成功启动
crc32	数据校验

那如何读取到 AvbABData 数据结构体呢? 这个数据体在 misc 分区中的起始位置由 AB_METADATA_OFFSET 宏来确定, 在我们的平台上, 这个宏等于 4, 但是这是 block 单位 (512 字节), 所以 $4 \times 512 = 2K$, 所以这个数据体就是在偏移 misc 分区头部 2K 的位置。

当读取到 AvbABData 数据体之后会进行校验, 已确定这玩意是不是 AvbABData, 如果不是、也就是校验失败, 它内部会构建这个数据体、并对其进行初始化操作。如果 misc 分区中不存在 AvbABData 数据, 则 SPL 会填充一个 (也就是初始化一个)

1.6.3 mmc_load_image_raw_sector 函数

```

static __maybe_unused
int mmc_load_image_raw_sector(struct spl_image_info *spl_image,
                             struct mmc *mmc, unsigned long sector)
{
    unsigned long count;
    struct image_header *header;    //描述了镜像头部信息
    int ret = 0;

    header = (struct image_header *) (CONFIG_SYS_TEXT_BASE -
                                       sizeof(struct image_header));

    /* read image header to find the image size & load address */
    count = blk_dread(mmc_get_blk_desc(mmc), sector, 1, header); //从分区的起始块中读取出一个块的数据
    debug("hdr read sector %lx, count=%lu\n", sector, count); //放入到header指向的内存中
    if (count == 0) {
        ret = -EIO;
        goto end;
    }

#ifdef CONFIG_SPL_FIT_IMAGE_MULTIPLE    //我们定义了这个宏=2
    if ((IS_ENABLED(CONFIG_SPL_LOAD_FIT) &&
        image_get_magic(header) == FDT_MAGIC) ||
        CONFIG_SPL_FIT_IMAGE_MULTIPLE > 1) { //这个条件也是成立的
#else
    if (IS_ENABLED(CONFIG_SPL_LOAD_FIT) &&
        image_get_magic(header) == FDT_MAGIC) {
#endif
        struct spl_load_info load;    //定义一个struct spl_load_info对象
        //定义这个对象的作用其实是因为接口的问题

        debug("Found FIT\n");    //将存储设备的读方法传递下去
        load.dev = mmc_get_blk_desc(mmc);
        load.priv = NULL;
        load.filename = NULL;
        load.bl_len = mmc->read_bl_len;    //这表示一个块的字节大小
        load.read = h_spl_load_read; //存储设备的读函数
        ret = spl_load_simple_fit(spl_image, &load, sector, header); //调用该函数加载镜像
    } else {
        ret = mmc_load_legacy(spl_image, mmc, sector, header);
    }

end:
    if (ret) {
#ifdef CONFIG_SPL_LIBCOMMON_SUPPORT
        debug("mmc_load_image_raw_sector error: ret is %d\n", ret);
#endif
        return -1;
    }

    return 0;
} ? end if (IS_ENABLED(CONFIG_SPL_LOAD_FIT) &&

```

调用 spl_load_simple_fit 的时候, 已经将 header 传递进去了。

CONFIG_SPL_FIT_IMAGE_MULTIPLE 在我们的 u-boot 中定义为 2, 它的作用:

uboot.img 在 uboot 分区中的偏移量, SPL 会从 uboot 分区的起始位置开始读取, 读取 uboot.img 镜像的头部, 然后进行校验, 确定这个头部是不是 uboot.img, 如果校验失败, SPL 会往后偏移一段 (偏移的大小应该是 CONFIG_SPL_FIT_IMAGE_KB), 然后再次读取 uboot.img 头部再进行校验。

CONFIG_SPL_FIT_IMAGE_MULTIPLE 这个应该是表示读取头部、校验的次数, 如果等于 1, 那么就直接读一次, 也就是直接从 uboot 分区起始位置读, 进行校验, 如果校验不对那就直接失败了。

1.6.4 spl_load_simple_fit 函数

```
int spl_load_simple_fit(struct spl_image_info *spl_image,
                        struct spl_load_info *info, ulong sector, void *fit)
{
    ulong sector_offs = sector;          // 记住 这是uboot分区的起始块
    int ret = -EINVAL;
    int i;

    for (i = 0; i < CONFIG_SPL_FIT_IMAGE_MULTIPLE; i++) {
        if (i > 0) { // 第一次for循环 不会进入这里
            sector_offs +=
                i * ((CONFIG_SPL_FIT_IMAGE_KB << 10) / info->bl_len); // 单位是块
            printf("Trying fit image at 0x%lx sector\n", sector_offs);
            if (info->read(info, sector_offs, 1, fit) != 1) {
                printf("IO error\n");
                continue; // 如果我们的uboot.img镜像就是直接放在uboot分区的起始地址 这样是
                           // 没有机会进入这个if的 所以sector_offs应该描述为uboot.img镜像所在位置的起始块
            }

            if (image_get_magic(fit) != FDT_MAGIC) {
                printf("Not fit magic\n");
                continue;
            }

            ret = spl_internal_load_simple_fit(spl_image, info, // 调用该函数
                                                sector_offs, fit);

            if (!ret) {
#ifdef CONFIG_SPL_KERNEL_BOOT
                ret = spl_load_kernel_fit(spl_image, info);
#endif
                break; // 如果spl_internal_load_simple_fit调用成功 则这里直接break退出 循环
            }
        } // end for i=0;i<CONFIG_SPL_FIT_IMAGE_MULTIPLE ?
#ifdef CONFIG_SPL_AB
        /*
         * If boot fail in spl, spl must decrease 1. If boot
         * successfully, it is no need to do that and U- boot will
         * always to decrease 1. If in thunderboot process,
         * always need to decrease 1.
         */
        if (IS_ENABLED(CONFIG_SPL_KERNEL_BOOT) || ret) // 上面成功的情况下 这里是不会执行的
            spl_ab_decrease_tries(info->dev);
#endif
        return ret;
    } // end spl_load_simple_fit ?
}
```

当第一次检验 uboot.img 头部失败时, 会往后偏移一段再读头部校验, 而偏移的大小就是这个:

sector_offs += ((CONFIG_SPL_FIT_IMAGE_KB << 10) / info->bl_len);

其实就是加上 CONFIG_SPL_FIT_IMAGE_KB (单位 KB), 当然你要转换为 block 块单位。

需要提一下的是, FIT 格式下, uboot.img 镜像头部校验魔数为 FDT_MAGIC (0xd00dfeed)。

1.6.5 spl_internal_load_simple_fit 函数

```

static int spl_internal_load_simple_fit(struct spl_image_info *spl_image,
                                       struct spl_load_info *info,
                                       ulong sector, void *fit_header)
{
    struct spl_image_info image_info; // 定义一个局部的spl_image_info对象 注意和传递进来的spl_image区分开
    int base_offset;
    int images, ret;
    int index = 0;
    int node = -1;
    void *fit;

    fit = spl_fit_load_blob(info, sector, fit_header, &base_offset); // 找到uboot.img镜像中的itb设备树镜像
    if (!fit) { // 并将其加载到内存中 其实uboot.img镜像头部就是设备树头
        debug("%s: Cannot load blob\n", __func__); // struct fdt_header 所以uboot.img镜像前面就是itb设备树镜像
        return -1;
    }

    /* find the node holding the images information */
    images = fdt_path_offset(fit, FIT_IMAGES_PATH); // 获取到itb设备树中的"/images"节点
    if (images < 0) {
        debug("%s: Cannot find /images node: %d\n", __func__, images);
        return -1;
    }

    /* if board sigs verify required, check self */
    if (fit_board_verify_required_sigs() && // 这是签名验证
        !IS_ENABLED(CONFIG_SPL_FIT_SIGNATURE)) {
        printf("Verified- boot requires CONFIG_SPL_FIT_SIGNATURE enabled\n");
        hang();
    }

    /* verify the configure node by keys, if required */
    #ifndef CONFIG_SPL_FIT_SIGNATURE // 没定义这个宏
    int conf_noffset;

    conf_noffset = fit_conf_get_node(fit, NULL);
    if (conf_noffset <= 0) {
        printf("No default config node\n");
        return -EINVAL;
    }

    ret = fit_config_verify(fit, conf_noffset);
    if (ret) {
        printf("fit verify configure failed, ret=%d\n", ret);
        return ret;
    }
    printf("\n");

    #ifndef CONFIG_SPL_FIT_ROLLBACK_PROTECT // 这个宏我们也没定义
    uint32_t this_index, min_index;

    ret = fit_rollback_index_verify(fit, FIT_ROLLBACK_INDEX_SPL,
                                    &this_index, &min_index);
    if (ret) {
        printf("fit failed to get rollback index, ret=%d\n", ret);
        return ret;
    }
    else if (this_index < min_index) {
        printf("fit reject rollback: %d < %d(min)\n",
               this_index, min_index);
        return -EINVAL;
    }
    #endif
}

```

```

    printf("rollback index: %d >= %d(min), OK\n", this_index, min_index);
#endif
#endif

/*
 * If required to start the other core before load "loadables"
 * firmwares, use the config "standalone" to load the other core's
 * firmware, then start it.
 * Normally, different cores' firmware is attach to the config
 * "loadables" and load them together.
 */
if (node < 0)
    node = spl_fit_get_image_node(fit, images, FIT_STANDALONE_PROP, // 找到itb设备树中
                                0); // configurations/conf节点中的"standalone"属性对应的节点
if (node > 0) { // 在我们的its设备树中, configurations/conf节点下并没有定义"standalone"属性
    /* Load the image and set up the spl_image structure */ // 所以这个if不会成立
    ret = spl_load_fit_image(info, sector, fit, base_offset, node,
                             &image_info);
    if (ret)
        return ret;

    if (image_info.entry_point == FDT_ERROR)
        image_info.entry_point = image_info.load_addr;

    ret = spl_fit_standalone_release(image_info.entry_point);
    if (ret)
        printf("Start standalone fail, ret = %d\n", ret);

    /* standalone is special one, continue to find others */
    node = - 1;
}

/*
 * Find the U- Boot image using the following search order:
 * - start at 'firmware' (e.g. an ARM Trusted Firmware)
 * - fall back 'kernel' (e.g. a Falcon- mode OS boot
 * - fall back to using the first 'loadables' entry
 */
if (node < 0)
    node = spl_fit_get_image_node(fit, images, FIT_FIRMWARE_PROP,
                                0); // 找configurations/conf节点中的"firmware"属性对应的节点 这个节点是有定义的 就是"atf-1"
#ifdef CONFIG_SPL_OS_BOOT // "atf-1" 所以node返回就是"atf-1"的节点
    if (node < 0) // CONFIG_SPL_OS_BOOT没定义
        node = spl_fit_get_image_node(fit, images, FIT_KERNEL_PROP, 0);
#endif
if (node < 0) { // 上面成功之后 这个node肯定是大于0 所以这里不会成立
    debug("could not find firmware image, trying loadables...\n");
    node = spl_fit_get_image_node(fit, images, "loadables", 0);
    /*
     * If we pick the U- Boot image from "loadables", start at
     * the second image when later loading additional images.
     */
    index = 1;
}
if (node < 0) {
    debug("%s: Cannot find u- boot image node: %d\n",
        __func__, node);
    return - 1;
}

/* Load the image and set up the spl_image structure */
ret = spl_load_fit_image(info, sector, fit, base_offset, node,
                         spl_image); // 到这里 已经把atf-1加载到了内存中 还没跳转 加载地址在spl_image变量中保存的

```

```

    if (ret)
        return ret;

    /*
     * For backward compatibility, we treat the first node that is
     * as a U-Boot image, if no OS- type has been declared.
     */
    if (! spl_fit_image_get_os(fit, node, &spl_image->os)) // 获取到atf-1节点的os属性 将其存放到spl_image->os中
        debug("Image OS is %s\n", genimg_get_os_name(spl_image->os));
    #if ! defined(CONFIG_SPL_OS_BOOT)
    else
        spl_image->os = IH_OS_U_BOOT;
    #endif

    /*
     * Booting a next- stage U-Boot may require us to append the FDT.
     * We allow this to fail, as the U-Boot image might embed its FDT.
     */
    if (spl_image->os == IH_OS_U_BOOT) // 如果spl_image->os=IH_OS_U_BOOT
        spl_fit_append_fdt(spl_image, info, sector, fit, // 我们的atf-1的os属性肯定不是这个
                           images, base_offset); // 所以不会执行

    /* Now check if there are more images for us to load */
    for (; index++) {
        uint8_t os_type = IH_OS_INVALID;

        node = spl_fit_get_image_node(fit, images, "loadables", index); // 找loadables属性中的各个节点
        if (node < 0) // 通过for循环遍历 index控制
            break;

        if (! spl_fit_image_get_os(fit, node, &os_type)) // 获取os属性
            debug("Loadable is %s\n", genimg_get_os_name(os_type));

        /* skip U-Boot ? */
        if (spl_image->next_stage == SPL_NEXT_STAGE_KERNEL &&
            os_type == IH_OS_U_BOOT) // 这个条件是不成立的
            continue;

        ret = spl_load_fit_image(info, sector, fit, base_offset, node,
                                &image_info); // 加载镜像到内存 加载地址会通过image_info返回出来
        if (ret < 0) // 通过这个for循环将loadables中列出的所有镜像加载到了对应的内存地址
            return ret;

        if (os_type == IH_OS_U_BOOT) { // 如果是uboot
            #if CONFIG_IS_ENABLED(ATF)
                spl_image->entry_point_bl33 = image_info.load_addr; // 将u-boot作为bl33 事实本就如此
            #elif CONFIG_IS_ENABLED(OPTEE)
                spl_image->entry_point_os = image_info.load_addr;
            #endif

            spl_fit_append_fdt(&image_info, info, sector, // 将uboot.img文件中携带的设备树镜像加载到u-boot镜像之后 内存
                              fit, images, base_offset); // 其实就是/configurations/conf节点中fdt属性指定的设备树镜像
            spl_image->fdt_addr = image_info.fdt_addr; // 将uboot设备树镜像的加载地址保存到spl_image->fdt_addr
        }

        /*
         * If the "firmware" image did not provide an entry point,
         * use the first valid entry point from the loadables.
         */
        if (spl_image->entry_point == FDT_ERROR &&
            image_info.entry_point != FDT_ERROR) // 这个不成立
            spl_image->entry_point = image_info.entry_point;
    }

```



```

/* Record our loadables into the FDT */
if (spl_image->fdt_addr) // 如果
    spl_fit_record_loadable(fit, images, index, // 这个其实就是在u-boot的设备树中创建"/fit-images"节点
                           spl_image->fdt_addr, // 然后填充一些镜像信息 spl_invoke_atf函数会去读取解析这个
                           &image_info);
} ? end for ;index++ ?

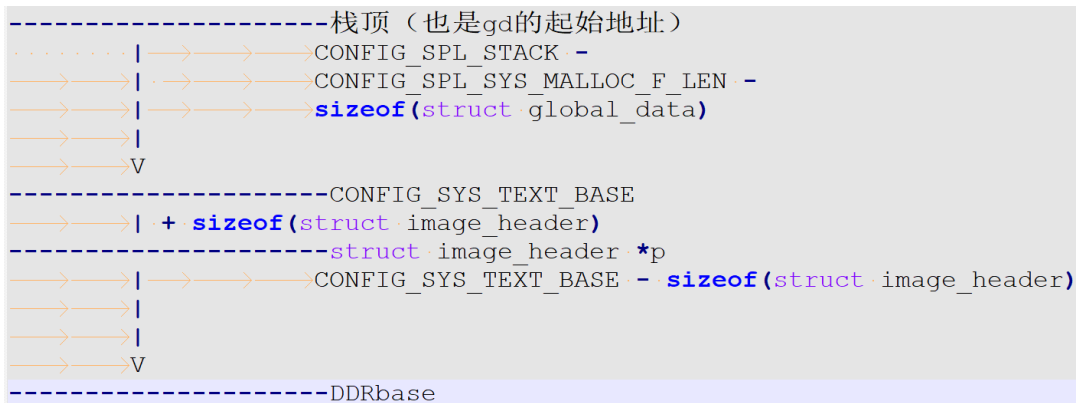
/*
 * If a platform does not provide CONFIG_SYS_UBOOT_START, U-Boot's
 * Makefile will set it to 0 and it will end up as the entry point
 * here. What it actually means is: use the load address.
 */
if (spl_image->entry_point == FDT_ERROR || spl_image->entry_point == 0)
    spl_image->entry_point = spl_image->load_addr;

return 0;
} ? end spl_internal_load_simple_fit ?

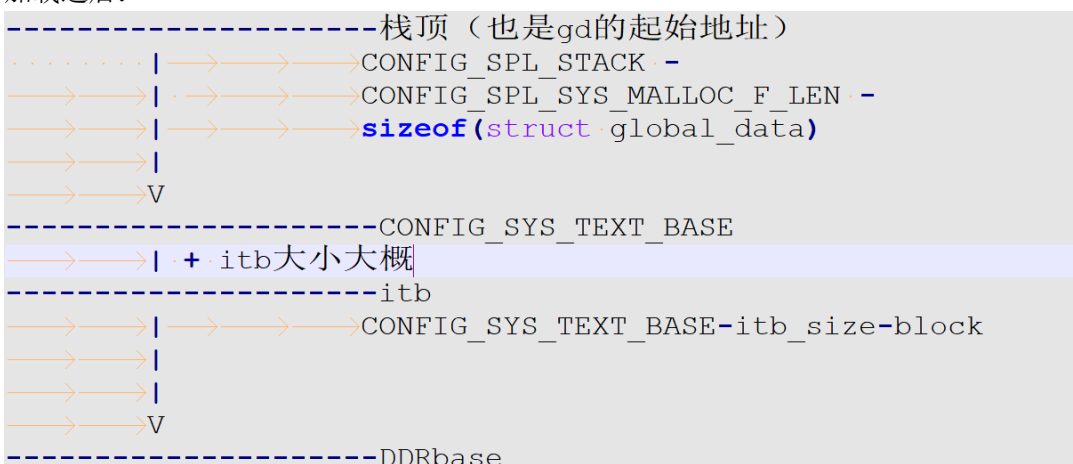
```

调用 `spl_internal_load_simple_fit` 函数传递进来的 `fit_header` 就是 `uboot.img` 镜像的头部, 其实就是 `its` 设备树镜像头部, 通过 `spl_fit_load_blob()` 函数将 `its` 设备树文件编译得到的二进制镜像 (`fdt_blob`) 加载到内存中, 位置就是 `CONFIG_SYS_TEXT_BASE` 往前的一段位置 (递减), 可能会把传递进来的 `fit_header` 给冲掉!

没加载之前是这样的:



加载之后:



`spl_fit_append_fdt` 函数加载了 `itb` 设备树中的 `/configurations/conf` 节点 `fdt` 属性指定的设备树镜像, 将其加载到了 `u-boot` 镜像之后, 也就是:

```

-----栈顶 (也是gd的起始地址)
.....|----->CONFIG_SPL_STACK -
----->|----->CONFIG_SPL_SYS_MALLOC_F_LEN -
----->|----->sizeof(struct global_data)
----->|
----->V

-----u-boot.dtb
----->| + u-boot.size
-----CONFIG_SYS_TEXT_BASE
----->| + itb大小大概
-----itb
----->|----->CONFIG_SYS_TEXT_BASE-itb_size-block
----->|
----->|
----->V

-----DDRbase

```

spl_fit_append_fdt 函数

```

static int spl_fit_append_fdt(struct spl_image_info *spl_image,
                             struct spl_load_info *info, ulong sector,
                             void *fit, int images, ulong base_offset)
{
    struct spl_image_info image_info;
    int node, ret;

    /* Figure out which device tree the board wants to use */
    node = spl_fit_get_image_node(fit, images, FIT_FDT_PROP, 0); //找到fdt属性中对应的节点(index=0)
    if (node < 0) {
        debug("%s: cannot find FDT node\n", __func__);
        return node;
    }

    /*
     * Read the device tree and place it after the image.
     * Align the destination address to ARCH_DMA_MINALIGN.
     */
    image_info.load_addr = spl_image->load_addr + spl_image->size;
    ret = spl_load_fit_image(info, sector, fit, base_offset, node,
                             &image_info); //加载uboot的设备树镜像 将其加载到u-boot镜像后面 紧跟u-boot镜像

    if (ret < 0)
        return ret;

    /* Make the load- address of the FDT available for the SPL framework */
    spl_image->fdt_addr = (void *)image_info.load_addr; //将u-boot设备树镜像加载地址保存在spl_image->fdt_addr中
    #if ! CONFIG_IS_ENABLED(FIT_IMAGE_TINY)
    /* Try to make space, so we can inject details on the loadables */
    ret = fdt_shrink_to_minimum(spl_image->fdt_addr, 8192);
    #endif

    /*
     * If need, load kernel FDT right after U-Boot FDT.
     *
     * kernel FDT is for U-Boot if there is not valid one
     * from images, ie: resource.img, boot.img or recovery.img.
     */
}

```

```

*/
node = spl_fit_get_image_node(fit, images, FIT_FDT_PROP, 1); // 这里是看能不能找到第二个设备树镜像 也就是fdt属性是否有多个值
if (node < 0) { // 如果存在 则将其作为内核设备树
    debug("%s: cannot find FDT node\n", __func__);
    return ret;
}

image_info.load_addr =
    (ulong)spl_image->fdt_addr + fdt_totalsize(spl_image->fdt_addr);
ret = spl_load_fit_image(info, sector, fit, base_offset, node, // 将内核设备树镜像的加载地址放在u-boot设备树镜像之后
    &image_info);

return ret;
} ? end spl_fit_append_fdt ?

```

1.6.6 spl_load_fit_image 函数

```

static int spl_load_fit_image(struct spl_load_info *info, ulong sector,
    void *fit, ulong base_offset, int node,
    struct spl_image_info *image_info)
{
    int offset;
    size_t length;
    int len;
    ulong size;
    ulong comp_addr, load_addr, load_ptr;
    void *src;
    ulong overhead;
    int nr_sectors;
    int align_len = ARCH_DMA_MINALIGN - 1;
    uint8_t image_comp = - 1, type = - 1;
    const void *data;
    bool external_data = false;

    if (IS_ENABLED(CONFIG_SPL_OS_BOOT) && IS_ENABLED(CONFIG_SPL_GZIP)) { // 这里不会成立
        if (fit_image_get_comp(fit, node, &image_comp))
            puts("Cannot get image compression format.\n");
        else
            debug("%s", genimg_get_comp_name(image_comp));

        if (fit_image_get_type(fit, node, &type))
            puts("Cannot get image type.\n");
        else
            debug("%s", genimg_get_type_name(type));
    } else {
        fit_image_get_comp(fit, node, &image_comp); // 获取压缩类型 compression属性
        // RK都没有进行压缩 所以compression=none
    }
}

```

```

if (fit_image_get_load(fit, node, &load_addr)) //获取load属性 load属性记录的是加载地址
    load_addr = image_info->load_addr;

if (image_comp != IH_COMP_NONE && image_comp != IH_COMP_ZIMAGE) {
    /* Empirically, 2MB is enough for U-Boot, tee and atf */
    if (fit_image_get_comp_addr(fit, node, &comp_addr)) //如果是压缩文件, 那么需要将压缩文件解压到加载地址
        comp_addr = load_addr + FIT_MAX_SPL_IMAGE_SZ;
} else {
    comp_addr = load_addr; //如果不是压缩文件, 直接拷贝到加载地址即可 我们的没有采用压缩
}

if (! fit_image_get_data_position(fit, node, &offset)) { //获取data- position属性 这个表示镜像在uboot.img文件的那个位置
    external_data = true; //表示itb文件中的镜像并没有包含在fdt_blob中 而是单独存放在itb中
} else if (! fit_image_get_data_offset(fit, node, &offset)) { //这是获取data- offset属性, 不过RK平台没有采用这种相对位置方式
    offset += base_offset; //而是采用了data- position属性表示的绝对位置
    external_data = true; //所以 如果是相对位置 那么data- offset表示的就是镜像相对于fdt_blob镜像末尾的偏移量
    //而传递进来的base_offset表示的就是fdt_blob镜像的大小
}

if (external_data) { //单独
    /* External data */
    if (fit_image_get_data_size(fit, node, &len)) //获取data- size属性 镜像的大小
        return - ENOENT;

    load_ptr = (comp_addr + align_len) & ~align_len; //加载地址确定 需要进行对齐操作
#if defined(CONFIG_ARCH_ROCKCHIP)
    if ((load_ptr < CONFIG_SYS_SDRAM_BASE) ||
        (load_ptr >= CONFIG_SYS_SDRAM_BASE + SDRAM_MAX_SIZE))
        load_ptr = (ulong)memalign(ARCH_DMA_MINALIGN, len);
#endif
    length = len; //确定拷贝长度

    overhead = get_aligned_image_overhead(info, offset);
    nr_sectors = get_aligned_image_size(info, length, offset);

    if (info->read(info, //从存储设备中读取镜像 到一个地址中 注意这里并不是直接读取到了加载地址
        sector + get_aligned_image_offset(info, offset), //因为涉及到存储设备存储单位block块的问题
        nr_sectors, (void *)load_ptr) != nr_sectors)
        return - EIO;

    debug("External data: dst=%lx, offset=%x, size=%lx\n",
        load_ptr, offset, (unsigned long)length);
    src = (void *)load_ptr + overhead; //这个地址就表示镜像在内存中的位置
} ? end if external_data ? else { //这种表示嵌入, 也就是说镜像是嵌入到fdt_blob中
    /* Embedded data */
    if (fit_image_get_data(fit, node, &data, &length)) { //获取镜像在内存中的地址
        puts("Cannot get image data/size\n"); //当这个地址并不是它的加载地址
        return - ENOENT;
    }
    debug("Embedded data: dst=%lx, size=%lx\n", load_addr,
        (unsigned long)length);
    src = (void *)data; //获取到镜像地址 src指向了镜像的地址 镜像是嵌入到了fdt_blob中
    //而fdt_blob前面已经加载到内存了 所以这些镜像同样也被加载到了内存中
}

/* Check hashes and signature */
if (image_comp != IH_COMP_NONE && image_comp != IH_COMP_ZIMAGE) //这里我们不成立
    printf("## Checking %s 0x%08lx (%s @0x%08lx) ... ",
        fit_get_name(fit, node, NULL), load_addr,
        (char *)fdt_getprop(fit, node, FIT_COMP_PROP, NULL),
        (long)src);
else // else分支成立
    printf("## Checking %s 0x%08lx ... ",

```

```

        fit_get_name(fit, node, NULL), load_addr);

#ifdef CONFIG_FIT_SPL_PRINT    //没定义
    printf("\n");
    fit_image_print(fit, node, "");
#endif
    if (! fit_image_verify_with_data(fit, node,    //这里就是核实 校验数据
                                     src, length)) //其实就是校验 hash
        return - EPERM;

#ifdef CONFIG_SPL_FIT_IMAGE_POST_PROCESS    //定义了这个宏
    board_fit_image_post_process(fit, node, (ulong *)&load_addr,
                                (ulong **)&src, &length);
#endif
    puts("OK\n");

    if (IS_ENABLED(CONFIG_SPL_OS_BOOT) &&    //这个if不成立
        IS_ENABLED(CONFIG_SPL_GZIP) &&
        image_comp == IH_COMP_GZIP &&
        type == IH_TYPE_KERNEL) {
        size = length;
        if (gunzip((void *)load_addr, CONFIG_SYS_BOOTM_LEN,
                  src, &size)) {
            puts("Uncompressing error\n");
            return - EIO;
        }
        length = size;
    } else {
        memcpy((void *)load_addr, src, length); //将镜像数据拷贝到它的加载地址
    }

    if (image_info) { //对image_info变量进行填充
        image_info->load_addr = load_addr;
        image_info->size = length;
        image_info->entry_point = fdt_getprop_u32(fit, node, "entry");
    }

    return 0;
} ? end spl_load_fit_image ?

```

RK 平台采用的是独立数据的方式，也就是镜像没有嵌入到 fdt_blob（its 设备树文件对应的二进制镜像文件）中，而是单独提供，与 fdt_blob 是平行关系放置在 uboot.img 镜像中。

对于嵌入的方式，在执行 spl_load_fit_image 函数之前，上层已经将 fdt_blob 加载到了内存中，由于镜像是嵌入的方式，所以，这些镜像也一并被加载到了内存中，所以只需找到它们在内存中的位置即可。

而对于独立的方式，镜像并不嵌入在 fdt_blob 中，所以需要单独再去从存储设备中读取出来，通过 data-position、data-size 等等这些属性来确定这些镜像在 uboot.img 镜像中位置，从而读取出来。

对于嵌入的方式，使用 fit_image_get_data 函数读取：

```

int fit_image_get_data(const void *fit, int noffset,
    const void **data, size_t *size)
{
    ulong data_off = 0;
    ulong data_pos = 0;
    int len;

    /* data */
    *data = fdt_getprop(fit, noffset, FIT_DATA_PROP, &len); // 获取data属性指定节点的内存位置
    if (*data) { // data指针直接指向了镜像的在内存中存储位置
        *size = len; // 获取成功直接返回
        return 0;
    }

    /* data- size */ // 如果上面失败 则进行下面的轴
    if (fit_image_get_data_size(fit, noffset, &len)) // 读取data- size属性 如果没有则会失败返回
        return - ENOENT;

    /* data- offset */
    if (! fit_image_get_data_offset(fit, noffset, (int *)&data_off)) { // 读取data- offset属性
        data_off += (ulong)fit + FIT_ALIGN(fdt_totalsize(fit)); // data- offset描述了镜像相对fdt_blob尾部的位置偏移量
        *data = (void *)data_off; // data_off = fdt_blob尾部的位置 + data- offset
        *size = len; // 成功直接返回
        return 0;
    }

    /* data- position */ // 如果上面获取data- offset属性失败 则会接着尝试读取data- position属性
    if (! fit_image_get_data_position(fit, noffset, (int *)&data_pos)) {
        *data = (void *) (data_pos + (ulong)fit); // data- position属性描述了镜像相对于fdt_blob头部的位置偏移量
        *size = len;
        return 0;
    }

    *size = 0;
    return - 1;
} ? end fit_image_get_data ?

```